

PROJET DATA SCIENCE - Bank Marketing Data Set

DEGNY GILLES ALFRED KOUTOUA CHRISTOPHER NIAMKE JOSEPH

```
# install.packages("xgboost")
# install.packages(c("dplyr", "readr", "ggplot2", "tidyr", "purrr", "tibble",
#                   "recipes", "themis", "glmnet", "ranger", "yardstick", "knitr"))

suppressPackageStartupMessages({
  library(dplyr)
  library(readr)
  library(ggplot2)
  library(tidyr)
  library(purrr)
  library(tibble)
  library(recipes)
  library(themis)
  library(glmnet)
  library(ranger)
  library(yardstick)
  library(knitr)
})
```

```
## Warning: le package 'recipes' a été compilé avec la version R 4.5.2
## Warning: le package 'themis' a été compilé avec la version R 4.5.3
## Warning: le package 'glmnet' a été compilé avec la version R 4.5.3
## Warning: le package 'ranger' a été compilé avec la version R 4.5.3
## Warning: le package 'yardstick' a été compilé avec la version R 4.5.3
## Warning: le package 'knitr' a été compilé avec la version R 4.5.3
```

ETAPE 1: COMPREHENSION METIER

OBJECTIF PRINCIPAL : Une banque a mené des campagnes téléphoniques pour convaincre ses clients de souscrire à un dépôt à terme. Notre objectif est de construire un modèle capable de prédire si un client va souscrire (**yes**) ou non (**no**).

OBJECTIF METIER : La banque veut utiliser les données des anciennes campagnes pour sélectionner les clients à cibler et les contacter par ordre de priorité, afin d'augmenter les souscriptions tout en limitant les coûts. On garde donc une approche relativement équilibrée.

1.1. Quelle est la variable cible? - Notre Variable cible est **y** - La variable cible **y** indique si un client a souscrit à un dépôt à terme (**yes** ou **no**) - L'intérêt métier : mieux cibler les clients susceptibles d'accepter l'offre, nous permettant ainsi de limiter les coûts.

1.2- A quel type de probleme avons-nous a faire: - C'est un problème de **classification binaire** car on prédit une variable à 2 catégories: **yes/no**

1.3- Que représente la campagne marketing? - Chaque ligne de nos données représente un appel téléphonique. La banque a donc appelé 41188 clients pour leur proposer l'offre. - un **yes** ou **no** dans la

variable cible signifie: - **yes** : le client a **accepté** l'offre - **no** : le client a **refusé** l'offre

ETAPE 2: COMPREHENSION DES DONNEES

Notre objectif ici sera de trouver les problemes liés aux données à savoir s'il y'a des données manquantes, des valeurs aberrantes et mieux connaître les différentes variables

Chargement des données

```
CHEMIN_FICHIER <- "bank-additional-full.csv"

Cestia <- read_delim(CHEMIN_FICHIER, delim = ";", show_col_types = FALSE)
head(Cestia)

## # A tibble: 6 x 21
##   age job   marital education default housing loan  contact month day_of_week
##   <dbl> <chr> <chr>   <chr>      <chr>    <chr>  <chr>  <chr>  <chr> <chr>
## 1   56 house~ married basic.4y  no      no      no    teleph~ may    mon
## 2   57 servi~ married high.sch~ unknown no      no    teleph~ may    mon
## 3   37 servi~ married high.sch~ no      yes    no    teleph~ may    mon
## 4   40 admin. married basic.6y  no      no      no    teleph~ may    mon
## 5   56 servi~ married high.sch~ no      no      yes    teleph~ may    mon
## 6   45 servi~ married basic.9y  unknown no      no    teleph~ may    mon
## # i 11 more variables: duration <dbl>, campaign <dbl>, pdays <dbl>,
## #   previous <dbl>, poutcome <chr>, emp.var.rate <dbl>, cons.price.idx <dbl>,
## #   cons.conf.idx <dbl>, euribor3m <dbl>, nr.employed <dbl>, y <chr>
```

2.1- la taille du dataset?

```
dim(Cestia)
```

```
## [1] 41188    21
```

- on dispose :
 - **41188** lignes
 - **21** colonnes

2.2- Les types de variables:

```
tibble(
  variable = names(Cestia),
  type = sapply(Cestia, function(x) class(x)[1])
)
```

```
## # A tibble: 21 x 2
##   variable      type
##   <chr>        <chr>
## 1 age          numeric
## 2 job          character
## 3 marital      character
## 4 education    character
## 5 default      character
## 6 housing      character
## 7 loan         character
## 8 contact      character
## 9 month        character
## 10 day_of_week character
## # i 11 more rows
```

- on dispose:
 - 10 variables **numériques**
 - 10 variables **catégorielles** + la variables **Cible**

2.3- les valeurs manquantes

```
colSums(is.na(Cestia))
```

```
##          age          job          marital          education          default
##           0           0           0           0           0
##       housing          loan          contact          month          day_of_week
##           0           0           0           0           0
##       duration          campaign          pdays          previous          poutcome
##           0           0           0           0           0
## emp.var.rate cons.price.idx cons.conf.idx          euribor3m          nr.employed
##           0           0           0           0           0
##           y
##           0
```

- Nous n'avons pas de données manquantes de types **NaN**

2.4- Verifions si nous disposons de valeurs manquantes de types unknown

```
compter_unknown <- sapply(Cestia, function(x) {
  if (is.character(x) || is.factor(x)) {
    sum(x == "unknown", na.rm = TRUE)
  } else {
    0
  }
})
```

```
compter_unknown
```

```
##          age          job          marital          education          default
##           0          330           80          1731          8597
##       housing          loan          contact          month          day_of_week
##          990          990           0           0           0
##       duration          campaign          pdays          previous          poutcome
##           0           0           0           0           0
## emp.var.rate cons.price.idx cons.conf.idx          euribor3m          nr.employed
##           0           0           0           0           0
##           y
##           0
```

- Nous disposons bien des valeurs manquantes de types **unknown** dans les variables:
 - job
 - marital
 - education
 - default
 - housing
 - loan
- **Décision:** Nous gardons **unknown** comme catégorie à part entière parce que **défaut** = 8597 soit environ **21%**. On ne peut donc pas supprimer ses lignes car nous nous exposons à de la perte d'informations.

2.5- verifions et affichons les doublons

```
nombre_doublons <- sum(duplicated(Cestia)); nombre_doublons
```

```
## [1] 12
```

- Notre dataset dispose de **12 doublons**

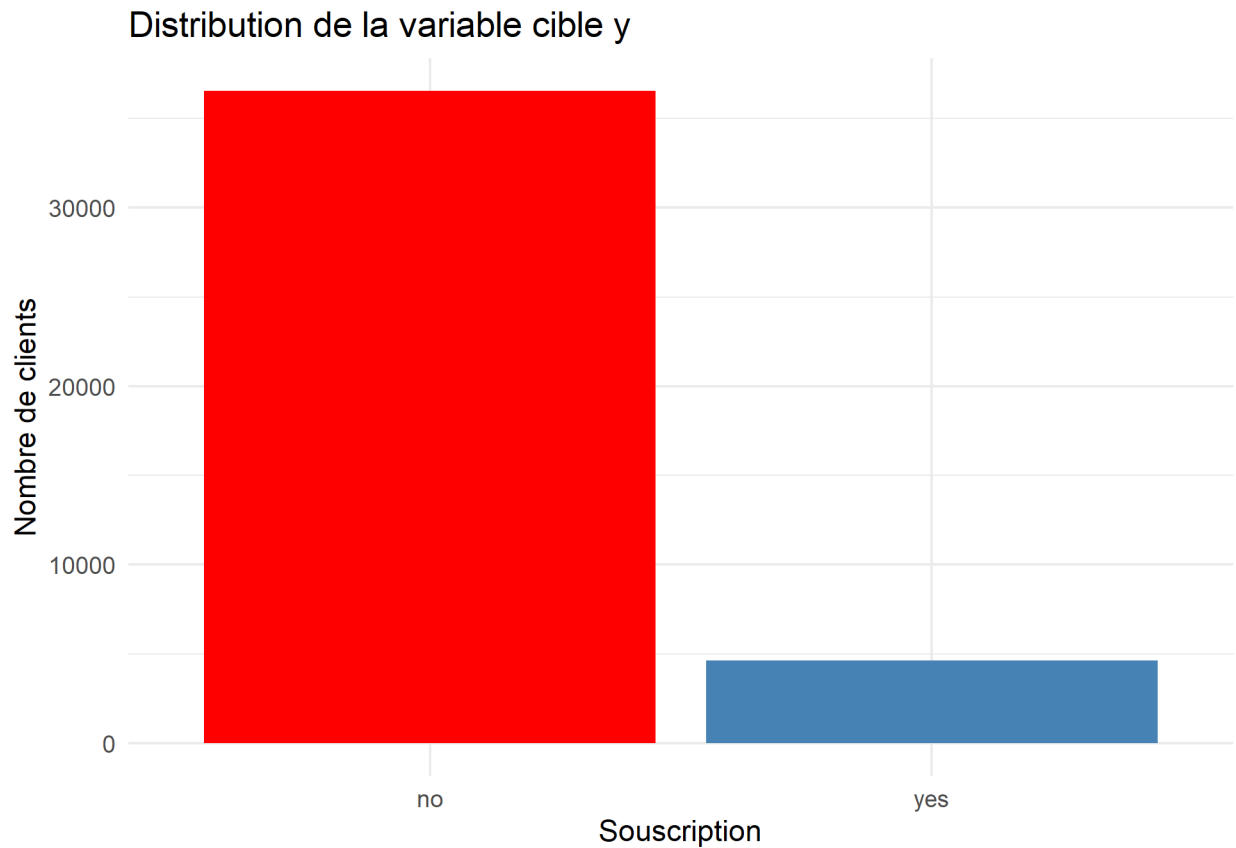
2.6- Distribution de la variable Cible y

```
count(Cestia, y, sort = TRUE)
```

```
## # A tibble: 2 x 2
##   y         n
##   <chr> <int>
## 1 no     36548
## 2 yes     4640
```

Graphique

```
count(Cestia, y) %>%
  ggplot(aes(x = y, y = n, fill = y)) +
  geom_col() +
  labs(
    title = "Distribution de la variable cible y",
    x = "Souscription",
    y = "Nombre de clients"
  ) +
  scale_fill_manual(values = c("no" = "red", "yes" = "steelblue")) +
  theme_minimal() +
  theme(legend.position = "none")
```



- **no** est 7,9 fois plus fréquent que **yes**
- Le dataset est donc **déséquilibré**. Cela veut dire que l'accuracy seule peut être trompeuse.
- Pour comparer les modèles, nous regarderons surtout la **PR-AUC**.

- Pour le choix du seuil, nous regarderons ensuite la **précision**, le **rappel** et le **F1-score**.
- Enfin, pour la valeur métier, nous regarderons le **Lift@20%** et le **Recall@20%**.

2.7. Vérifions l'existence des valeurs aberrantes dans le jeu de données

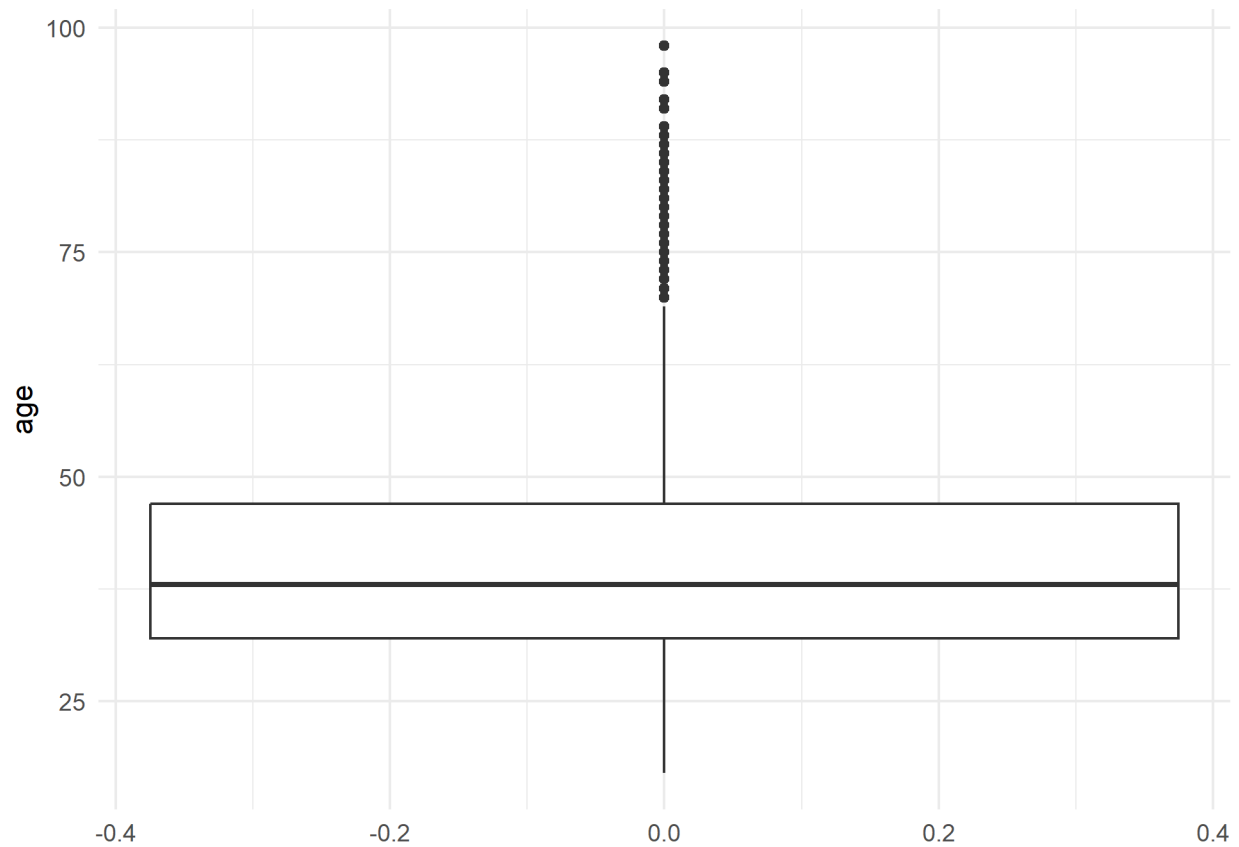
Statistique descriptive

```
summary(select(Cestia, where(is.numeric)))
```

```
##      age      duration      campaign      pdays
## Min.   :17.00   Min.    :  0.0   Min.    : 1.000   Min.    :  0.0
## 1st Qu.:32.00   1st Qu.: 102.0   1st Qu.: 1.000   1st Qu.:999.0
## Median :38.00   Median : 180.0   Median : 2.000   Median :999.0
## Mean   :40.02   Mean    : 258.3   Mean    : 2.568   Mean    :962.5
## 3rd Qu.:47.00   3rd Qu.: 319.0   3rd Qu.: 3.000   3rd Qu.:999.0
## Max.    :98.00   Max.    :4918.0   Max.    :56.000   Max.    :999.0
## previous emp.var.rate cons.price.idx cons.conf.idx
## Min.    :0.000   Min.    :-3.40000   Min.    :92.20   Min.    : -50.8
## 1st Qu.:0.000   1st Qu.: -1.80000   1st Qu.:93.08   1st Qu.: -42.7
## Median :0.000   Median :  1.10000   Median :93.75   Median : -41.8
## Mean    :0.173   Mean    :  0.08189   Mean    :93.58   Mean    : -40.5
## 3rd Qu.:0.000   3rd Qu.:  1.40000   3rd Qu.:93.99   3rd Qu.: -36.4
## Max.    :7.000   Max.    :  1.40000   Max.    :94.77   Max.    : -26.9
## euribor3m nr.employed
## Min.    :0.634   Min.    :4964
## 1st Qu.:1.344   1st Qu.:5099
## Median :4.857   Median :5191
## Mean    :3.621   Mean    :5167
## 3rd Qu.:4.961   3rd Qu.:5228
## Max.    :5.045   Max.    :5228
```

ÂGE

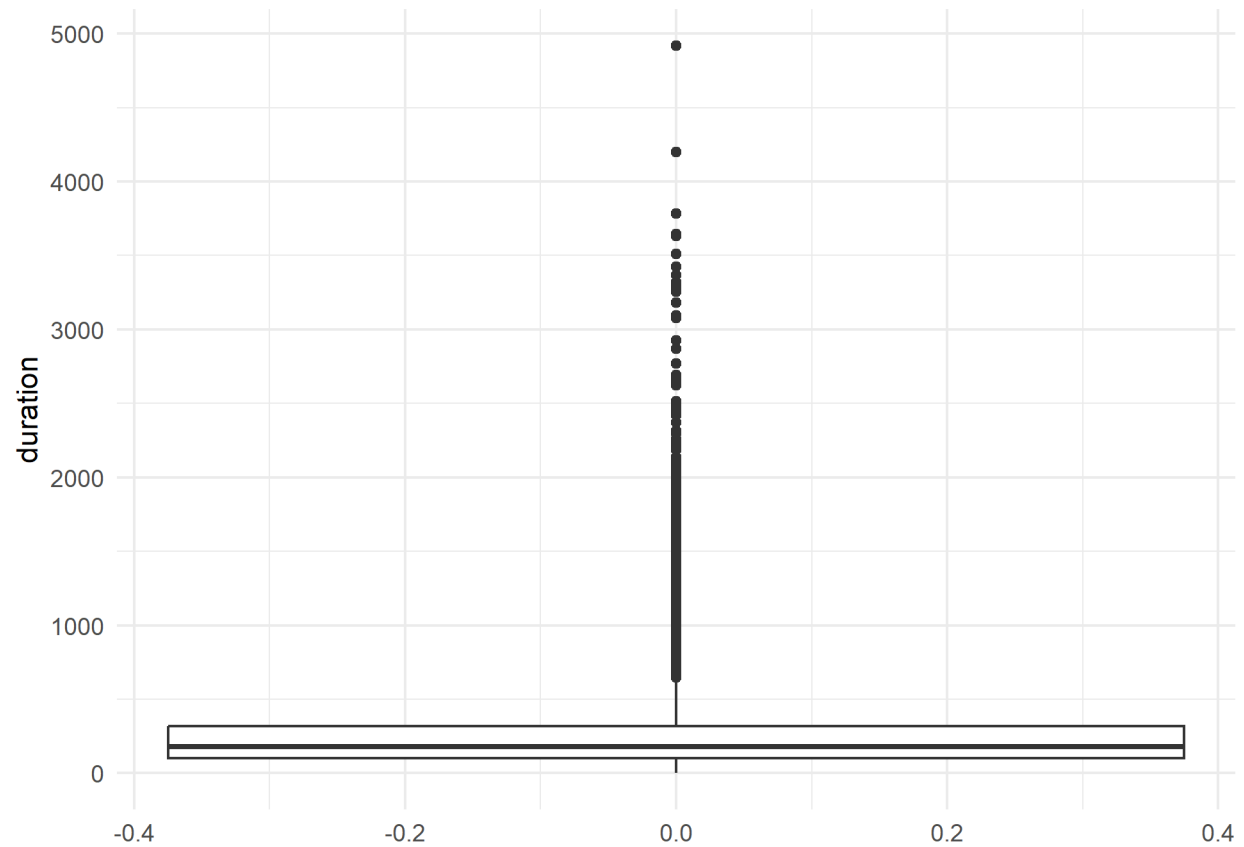
```
ggplot(Cestia, aes(y = age)) +
  geom_boxplot() +
  labs(y = "age") +
  theme_minimal()
```



- **ÂGE:**
 - l'âge minimum d'un client est **17** et l'âge maximum est **98**
 - la majorité des clients ont entre **32** et **47** ans
 - la Variable **Âge** a **469** valeurs aberrantes

DURATION

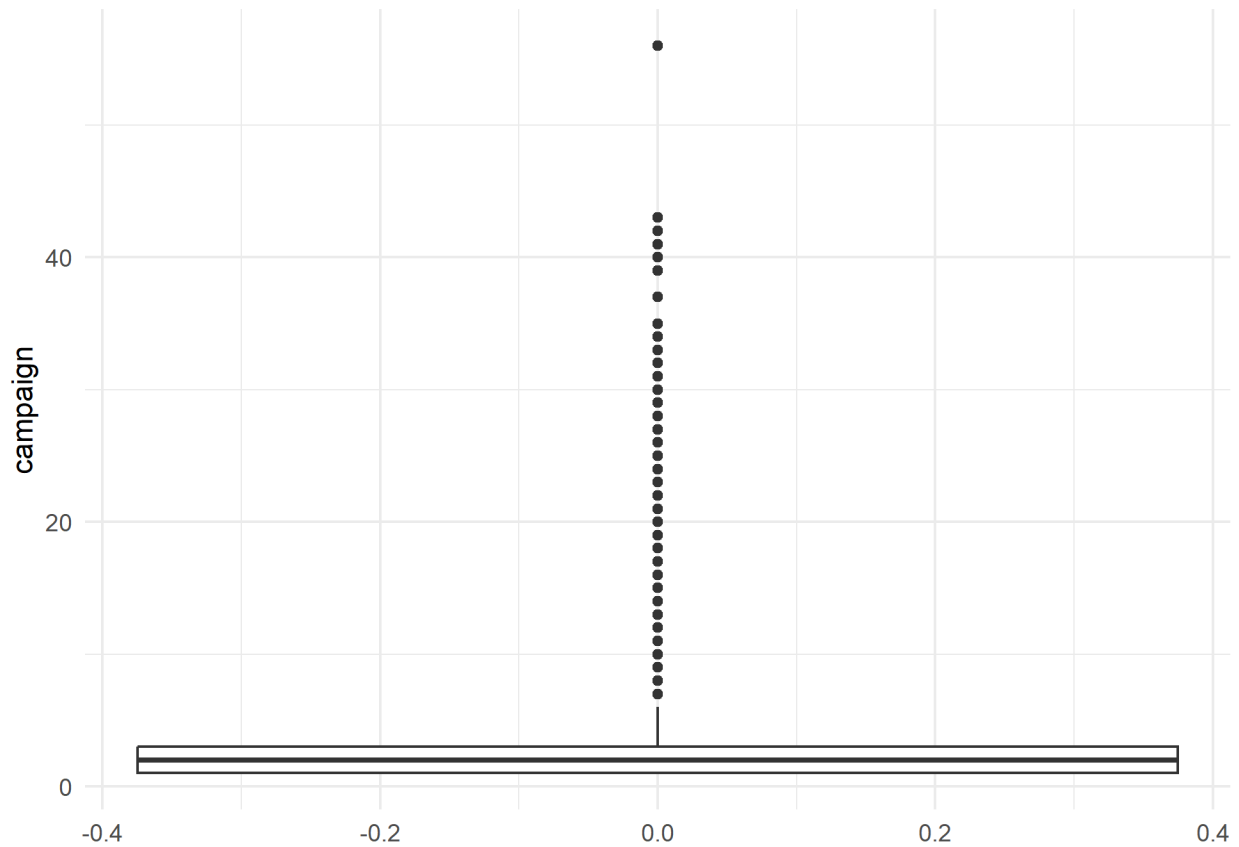
```
ggplot(Cestia, aes(y = duration)) +  
  geom_boxplot() +  
  labs(y = "duration") +  
  theme_minimal()
```



- **DURATION:**
 - 75% des appels durent moins de **319** secondes
 - mais un appel peut durer **4918**
 - On devra supprimer la variable duration car elle est connue qu'après un appel

CAMPAIGN

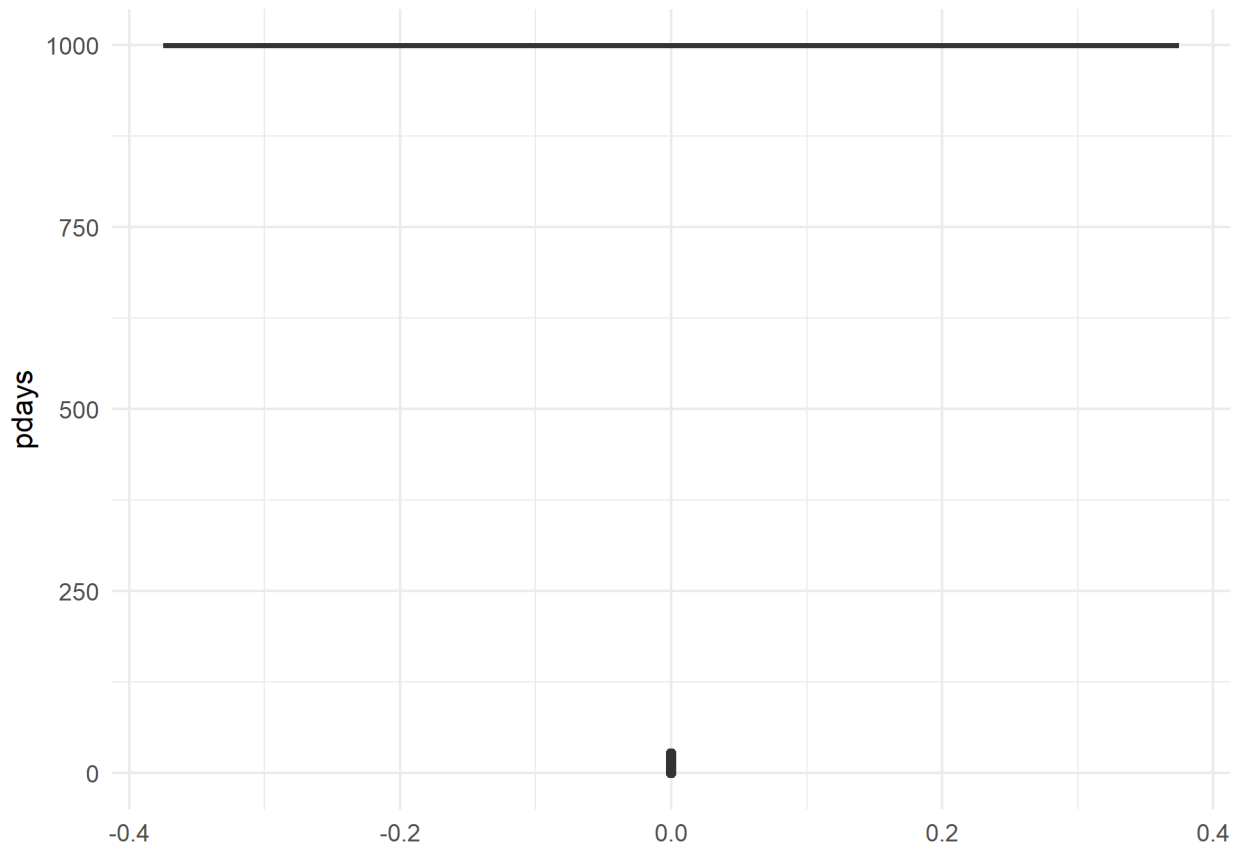
```
ggplot(Cestia, aes(y = campaign)) +
  geom_boxplot() +
  labs(y = "campaign") +
  theme_minimal()
```



- **campaign:**
 - la plupart des clients sont contactés **1 à 3 fois**
 - mais certains **56 fois**
 - **56 est une valeur aberrante**

PDAYS

```
ggplot(Cestia, aes(y = pdays)) +
  geom_boxplot() +
  labs(y = "pdays") +
  theme_minimal()
```

- pdays = 999 : signifie “jamais contacté avant” . Ce n’est pas une vraie valeur numérique
- 39673 clients ont cette valeur. À traiter comme catégorie ou indicateur binaire

2.8- Coorelation des variables explicatives avec la variable cible

```
Cestia <- Cestia %>% mutate(y_numerique = if_else(y == "yes", 1, 0))
```

Colonnes numériques

```
colonnes_numerique <- names(Cestia)[sapply(Cestia, is.numeric)]
colonnes_numerique <- setdiff(colonnes_numerique, "y_numerique")
colonnes_numerique
```

```
## [1] "age"          "duration"      "campaign"      "pdays"
## [5] "previous"     "emp.var.rate"  "cons.price.idx" "cons.conf.idx"
## [9] "euribor3m"    "nr.employed"
```

Correlation avec la variable cible

```
matrice_corr <- Cestia %>%
  select(all_of(c(colonnes_numerique, "y_numerique"))) %>%
  cor(use = "complete.obs")

correlation <- matrice_corr[rownames(matrice_corr) != "y_numerique", "y_numerique"]
correlation
```

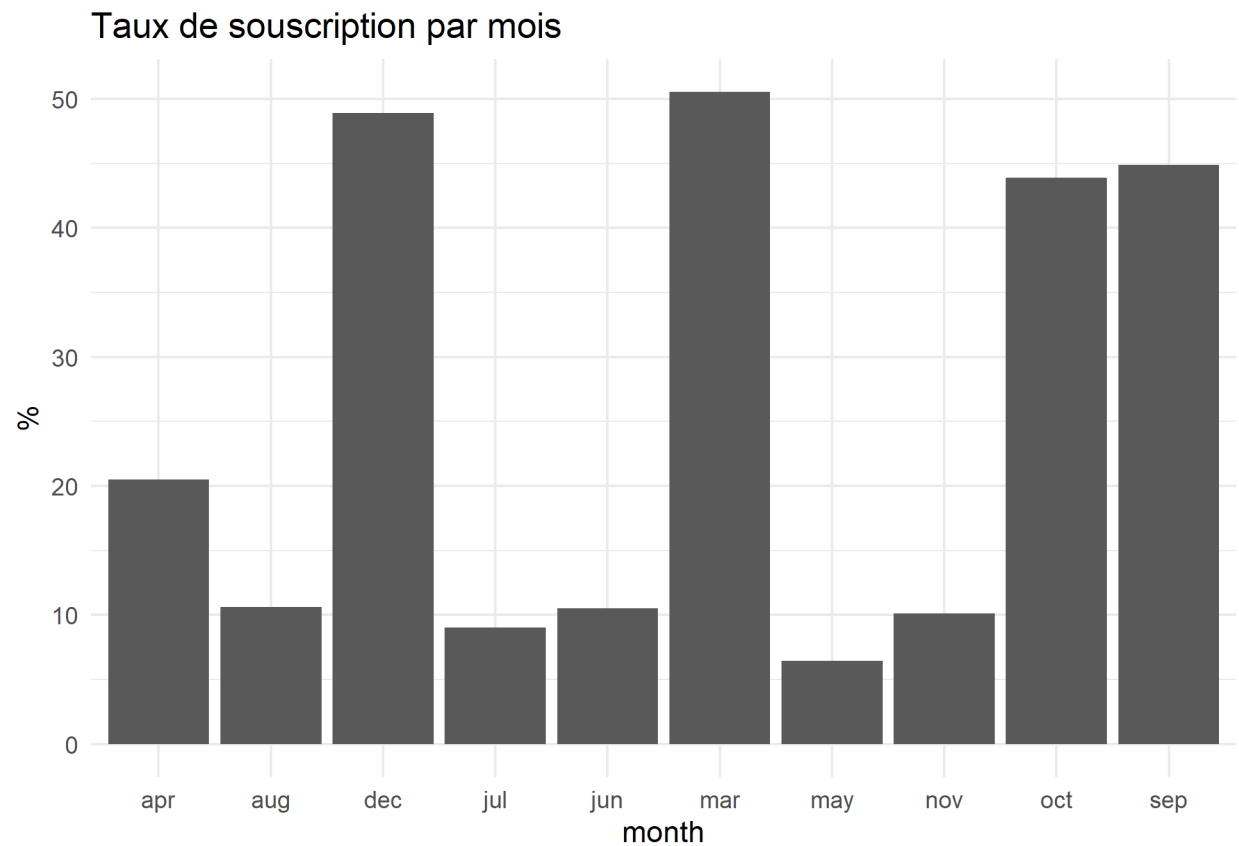
```
##      age      duration      campaign      pdays      previous
## 0.03039880 0.40527380 -0.06635741 -0.32491448 0.23018100
## emp.var.rate cons.price.idx cons.conf.idx euribor3m nr.employed
## -0.29833443 -0.13621121 0.05487795 -0.30777140 -0.35467830
```

Taux de souscription par mois

```
taux_mois <- Cestia %>%  
  group_by(month) %>%  
  summarise(taux = mean(y_numerique), .groups = "drop")  
  
print(taux_mois %>% mutate(taux = round(taux * 100, 2)))
```

```
## # A tibble: 10 x 2  
##   month  taux  
##   <chr> <dbl>  
## 1 apr    20.5  
## 2 aug    10.6  
## 3 dec    48.9  
## 4 jul     9.05  
## 5 jun    10.5  
## 6 mar    50.6  
## 7 may     6.43  
## 8 nov    10.1  
## 9 oct    43.9  
## 10 sep   44.9
```

```
taux_mois %>%  
  ggplot(aes(x = month, y = taux * 100)) +  
  geom_col() +  
  labs(title = "Taux de souscription par mois", y = "%", x = "month") +  
  theme_minimal()
```



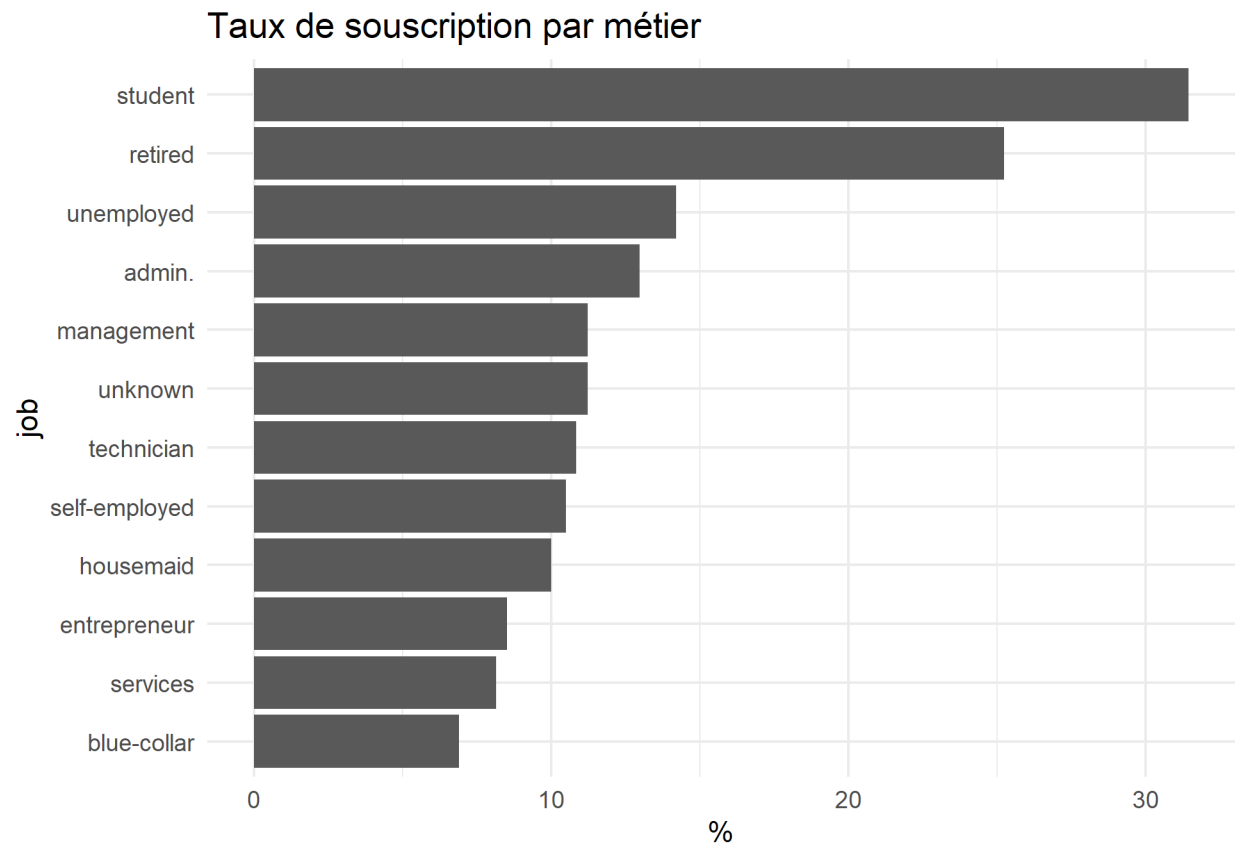
Taux de souscription par metier

```
taux_metier <- Cestia %>%
  group_by(job) %>%
  summarise(taux = mean(y_numerique), .groups = "drop")

print(taux_metier %>% mutate(taux = round(taux * 100, 2)))

## # A tibble: 12 x 2
##   job      taux
##   <chr>    <dbl>
## 1 admin.    13.0
## 2 blue-collar  6.89
## 3 entrepreneur  8.52
## 4 housemaid    10
## 5 management  11.2
## 6 retired    25.2
## 7 self-employed 10.5
## 8 services    8.14
## 9 student    31.4
## 10 technician  10.8
## 11 unemployed  14.2
## 12 unknown    11.2

taux_metier %>%
  ggplot(aes(x = taux * 100, y = reorder(job, taux))) +
  geom_col() +
  labs(title = "Taux de souscription par métier", x = "%", y = "job") +
  theme_minimal()
```

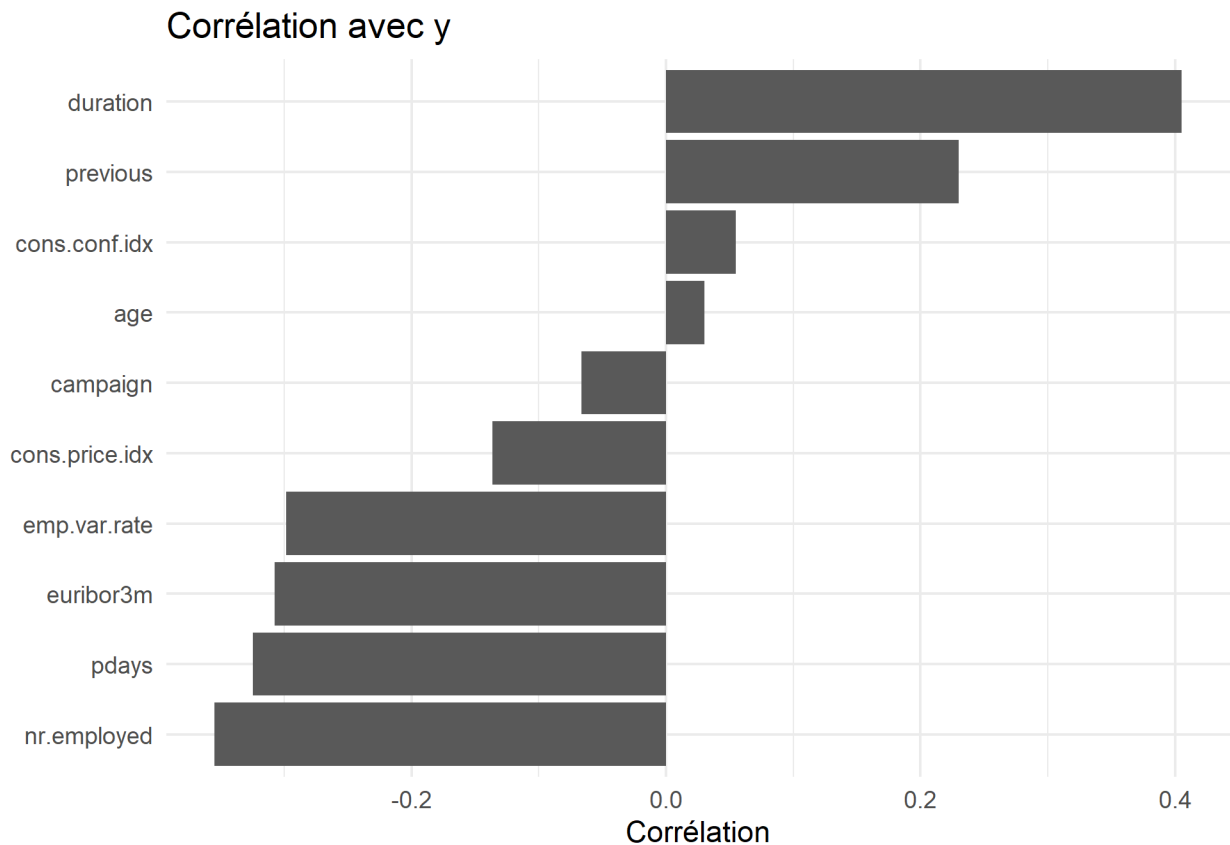


correlation avec la variable cible

```
cor_df <- tibble(
  variable = names(correlation),
  correlation = as.numeric(correlation)
) %>%
  filter(!is.na(correlation)) %>%
  arrange(correlation)

cor_df$variable <- factor(cor_df$variable, levels = cor_df$variable)

ggplot(cor_df, aes(x = correlation, y = variable)) +
  geom_col() +
  labs(title = "Corrélation avec y", x = "Corrélation", y = NULL) +
  theme_minimal()
```

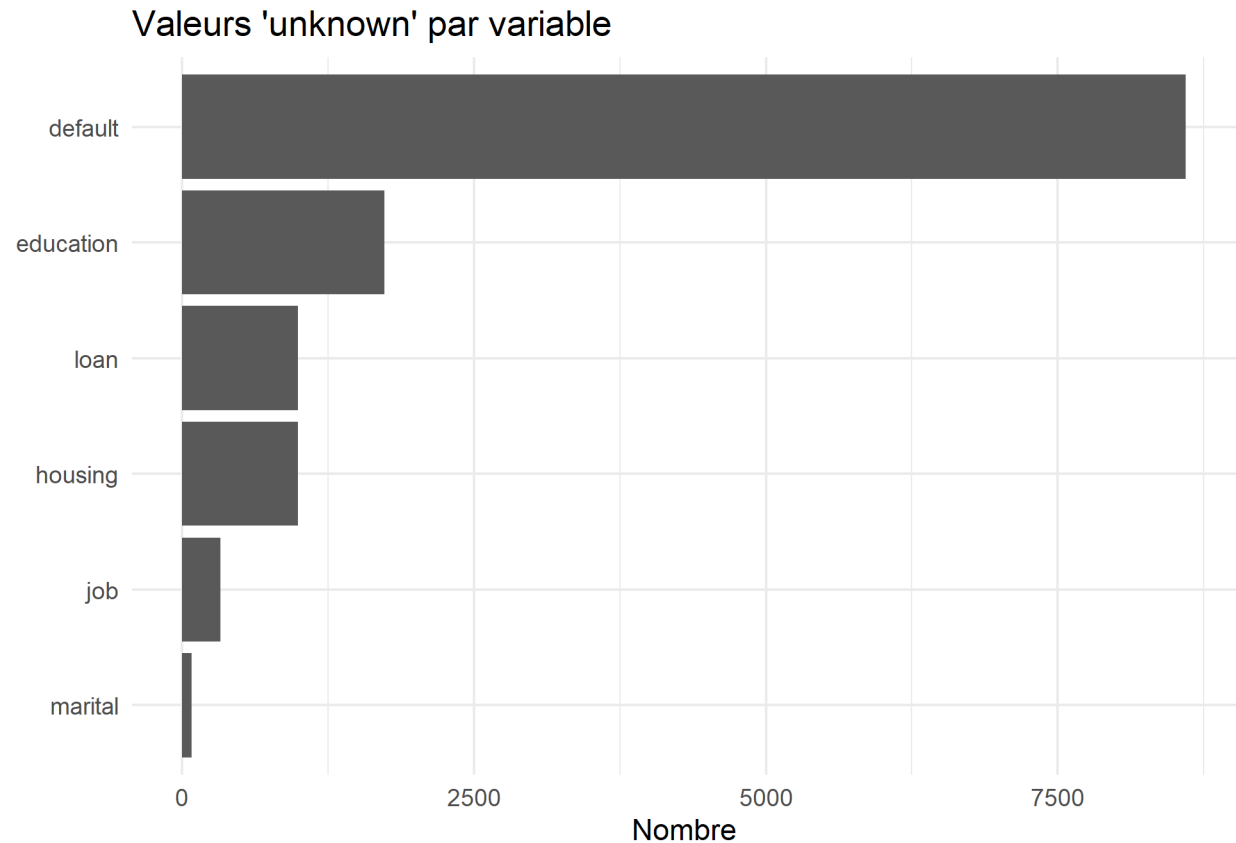


visualisation de la variable unknown

```
unknown <- sapply(Cestia %>% select(where(~ is.character(.x) || is.factor(.x))), function(x) {
  sum(x == "unknown", na.rm = TRUE)
})

unknown <- unknown[unknown > 0]

tibble(
  variable = names(unknown),
  n = as.numeric(unknown)
) %>%
  ggplot(aes(x = n, y = reorder(variable, n))) +
  geom_col() +
  labs(title = "Valeurs 'unknown' par variable", x = "Nombre", y = NULL) +
  theme_minimal()
```



Conclusion partielle: - Nous gardons 'unknown' comme catégorie à cause de **default** = 8597)

- Valeurs aberrantes :
 - Supprimer la variable **duration** car elle crée une fuite de données
 - Créer une variable binaire **contacted_before** et une variable **pdays_clean** pour mieux traiter **pdays**
- La variable cible étant déséquilibrée, on utilise F1-score pour un compromis entre précision et rappel, et PR-AUC pour la sélection du modèle.
- Variables importantes identifiées :
 - duration (*correlation* = 0.41)
 - nr.employed (*correlation* = -0.36)
 - euribor3m (*correlation* = -0.31)
 - mars, décembre, septembre et octobre ont un **taux > 40%**
 - taux de souscription par job : **student (31%), retired (25%)**

ETAPE 3: PREPARATION DES DONNEES

3.1 – Repartons du dataset brut

```
Cestia_modele <- Cestia
cat("Taille initiale :", dim(Cestia_modele), "\n")
```

```
## Taille initiale : 41188 22
```

3.2 – Supprimons les doublons

```
cat("Lignes avant suppression des doublons :", nrow(Cestia_modele), "
")
```

```
## Lignes avant suppression des doublons : 41188
```

```
Cestia_modele <- distinct(Cestia_modele)
cat("Lignes après suppression des doublons :", nrow(Cestia_modele), "
")
```

```
## Lignes après suppression des doublons : 41176
```

3.3 – Supprimons les variables qui posent problème pour une prédiction avant la campagne

```
colonnes_a_supprimer <- c("duration", "campaign")
Cestia_modele <- Cestia_modele %>% select(-all_of(colonnes_a_supprimer))
cat("Colonnes supprimées :", paste(colonnes_a_supprimer, collapse = ", "), "
")
```

```
## Colonnes supprimées : duration, campaign
```

```
cat("Nouvelle taille :", dim(Cestia_modele), "
")
```

```
## Nouvelle taille : 41176 20
```

- **duration** est connue après l'appel, donc elle crée une fuite de données.
- **campaign** dépend de la campagne en cours, donc elle n'est pas cohérente avec notre objectif de prédiction avant l'action commerciale.

3.4 – Transformons la variable pdays

```
Cestia_modele <- Cestia_modele %>%
  mutate(
    contacted_before = if_else(pdays != 999, 1, 0),
    pdays_clean = if_else(pdays < 999, pdays, 999)
  ) %>%
  select(-pdays)

head(Cestia_modele %>% select(contacted_before, pdays_clean))
```

```
## # A tibble: 6 x 2
##   contacted_before pdays_clean
##           <dbl>         <dbl>
## 1             0           999
## 2             0           999
## 3             0           999
## 4             0           999
## 5             0           999
## 6             0           999
```

```
count(Cestia_modele, contacted_before)
```

```
## # A tibble: 2 x 2
##   contacted_before     n
##           <dbl> <int>
## 1             0 39661
## 2             1 1515
```

- Ici, **999** signifie en réalité “jamais contacté auparavant”.
- On transforme donc **pdays** en deux informations plus lisibles :

- `contacted_before` : le client a-t-il déjà été contacté ?
- `pdays_clean` : nombre de jours si l'information existe.

3.5 – Encodons la variable cible y

```
Cestia_modele <- Cestia_modele %>%
  mutate(y = if_else(y == "yes", 1, 0))

cat("Répartition finale de la cible :
")
```

```
## Répartition finale de la cible :
```

```
print(count(Cestia_modele, y))
```

```
## # A tibble: 2 x 2
##       y         n
##   <dbl> <int>
## 1     0 36537
## 2     1 4639
```

```
cat("
Proportions :
")
```

```
##
```

```
## Proportions :
```

```
print(prop.table(table(Cestia_modele$y)) %>% round(4))
```

```
##
```

```
##           0           1
## 0.8873 0.1127
```

3.6 – Regroupons les mois en saisons

```
saison_temporelle <- c(
  "mar" = "printemps", "apr" = "printemps", "may" = "printemps",
  "jun" = "ete", "jul" = "ete", "aug" = "ete",
  "sep" = "automne", "oct" = "automne", "nov" = "automne",
  "dec" = "hiver", "jan" = "hiver", "feb" = "hiver"
)
```

```
Cestia_modele <- Cestia_modele %>%
  mutate(saison = recode(month, !!!saison_temporelle))
```

```
count(Cestia_modele, saison, sort = TRUE)
```

```
## # A tibble: 4 x 2
##   saison         n
##   <chr>      <int>
## 1 ete      18663
## 2 printemps 16944
## 3 automne   5387
## 4 hiver     182
```

- Cette transformation simplifie la variable **month**.
- Elle permet de garder l'idée du moment de l'année, tout en réduisant le nombre de catégories.

3.7 – Regroupons les niveaux d'éducation


```

education_niveaux <- c(
  "illiterate" = "bas",
  "basic.4y" = "bas",
  "basic.6y" = "bas",
  "basic.9y" = "moyen",
  "high.school" = "moyen",
  "professional.course" = "moyen",
  "university.degree" = "haut",
  "unknown" = "unknown"
)

Cestia_modele <- Cestia_modele %>%
  mutate(education_niveau = recode(education, !!!education_niveaux))

count(Cestia_modele, education_niveau, sort = TRUE)

```

```

## # A tibble: 4 x 2
##   education_niveau      n
##   <chr>            <int>
## 1 moyen            20797
## 2 haut             12164
## 3 bas              6485
## 4 unknown         1730

```

- Cela rend la variable plus simple à lire.
- Cela évite aussi d'avoir trop de petites modalités séparées.

3.8 – Choisissons les variables finales

```

colonnes_finales <- c(
  "age",
  "job",
  "marital",
  "education_niveau",
  "default",
  "housing",
  "loan",
  "contact",
  "saison",
  "day_of_week",
  "previous",
  "poutcome",
  "emp.var.rate",
  "cons.price.idx",
  "cons.conf.idx",
  "euribor3m",
  "nr.employed",
  "contacted_before",
  "pdays_clean",
  "y"
)

Cestia_modele <- Cestia_modele %>% select(all_of(colonnes_finales))
cat("Taille du dataset final de modélisation :", dim(Cestia_modele), "
")

```

```
## Taille du dataset final de modélisation : 41176 20
```

```
head(Cestia_modele)
```

```
## # A tibble: 6 x 20
##   age job      marital education_niveau default housing loan  contact  saison
##   <dbl> <chr>    <chr>    <chr>          <chr>  <chr>  <chr> <chr>    <chr>
## 1   56 housemaid married bas              no    no    no    telepho~ print~
## 2   57 services married moyen            unknown no    no    telepho~ print~
## 3   37 services married moyen              no    yes   no    telepho~ print~
## 4   40 admin.   married bas              no    no    no    telepho~ print~
## 5   56 services married moyen              no    no    yes   telepho~ print~
## 6   45 services married moyen            unknown no    no    telepho~ print~
## # i 11 more variables: day_of_week <chr>, previous <dbl>, poutcome <chr>,
## #   emp.var.rate <dbl>, cons.price.idx <dbl>, cons.conf.idx <dbl>,
## #   euribor3m <dbl>, nr.employed <dbl>, contacted_before <dbl>,
## #   pdays_clean <dbl>, y <dbl>
```

- Pour la version finale, on garde les variables utiles avant campagne.
- On remplace **month** par **saison** et **education** par **education_niveau** pour simplifier le jeu de données.

3.9 – Séparons les données dans l'ordre du temps : train / validation / test

```
n <- nrow(Cestia_modele)
fin_train <- floor(0.70 * n)
fin_valid <- floor(0.85 * n)

train <- Cestia_modele[1:fin_train, , drop = FALSE]
valid <- Cestia_modele[(fin_train + 1):fin_valid, , drop = FALSE]
test <- Cestia_modele[(fin_valid + 1):n, , drop = FALSE]

cat("Train :", dim(train), "
")
```

```
## Train : 28823 20
```

```
cat("Validation :", dim(valid), "
")
```

```
## Validation : 6176 20
```

```
cat("Test :", dim(test), "
")
```

```
## Test : 6177 20
```

- Ici, on **ne mélange pas les lignes**.
- Le but est de respecter l'ordre temporel du dataset : on apprend sur le passé, puis on valide sur des observations plus récentes, et on garde le test pour la fin.

3.10 – Séparons X et y

```
X_train <- train %>% select(-y)
y_train <- train$y

X_valid <- valid %>% select(-y)
y_valid <- valid$y

X_test <- test %>% select(-y)
y_test <- test$y

cat("Taille X_train :", dim(X_train), "
")
```

```
## Taille X_train : 28823 19
```

```
cat("Taille X_valid :", dim(X_valid), "  
")
```

```
## Taille X_valid : 6176 19
```

```
cat("Taille X_test  :", dim(X_test), "  
")
```

```
## Taille X_test   : 6177 19
```

3.11 – Définissons les colonnes numériques et catégorielles

```
colonnes_numeriques <- c(  
  "age",  
  "previous",  
  "emp.var.rate",  
  "cons.price.idx",  
  "cons.conf.idx",  
  "euribor3m",  
  "nr.employed",  
  "contacted_before",  
  "pdays_clean"  
)
```

```
colonnes_categorielles <- c(  
  "job",  
  "marital",  
  "education_niveau",  
  "default",  
  "housing",  
  "loan",  
  "contact",  
  "saison",  
  "day_of_week",  
  "poutcome"  
)
```

```
cat("Nombre de variables numériques :", length(colonnes_numeriques), "  
")
```

```
## Nombre de variables numériques : 9
```

```
cat("Nombre de variables catégorielles :", length(colonnes_categorielles), "  
")
```

```
## Nombre de variables catégorielles : 10
```

ETAPE 4: ENTRAINER LES MODELES

Dans cette partie, nous allons parlerons des modèles choisis et nous les comparerons.

Les modèles choisis

Les modèles retenus pour notre études sont les suivants: - La **regression logistique** - **Random Forest** - **XGBOOST**

Le choix de ces trois algorithmes permet de couvrir le large spectre de la modélisation statistique.

Étant face à un problème de **classification binaire**, l'idée est de commencer avec une **approche linéaire (regression logistique)** pour sa simplicité et son interprétabilité.

Par la suite, nous évaluerons les approches par **agrégation parallèle (Random forest)** et par **optimisation séquentielle (XGBOOST)**;

L'approche par agrégation parallèle ayant pour particularité d'être stable et robuste face au bruit, et celle par optimisation séquentielle la particularité d'être performante avec des données tabulaires.

4.1 – Régression logistique

Définition :

La régression logistique est un modèle linéaire de classification. Elle ne prédit pas directement une classe **no** ou **yes**, mais la probabilité qu'un événement appartienne à la classe **yes**.

Elle se fait en deux étapes: - **étape 1 : le score linéaire (z)** . On calcule d'abord une somme pondérée des variables:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

où $x_1, x_2, \dots$ sont les données d'entrée (*features*) et β_0 est la valeur moyenne estimée de la variable z lorsque les variables explicatives sont nulles; $\beta_1, \beta_2, \dots$ sont les poids.

- **étape 2 : La fonction Sigmoïde σ** . On transforme ce score z en une probabilité entre 0 et 1 grâce à la fonction logistique :

$$\sigma(z) = \frac{1}{1 + e^{-z}} = P(y = 1)$$

y étant la valeur de la cible **0** pour **non/no** et **1** pour **oui/yes**.

Comment elle est appliquée dans notre cas?

Nous allons faire la régression logistique de deux façons:

- Régression logistique avec classes équilibrées

Ici, les variables numériques sont d'abord standardisées. Ensuite, les variables catégorielles sont encodées en One Hot Encoding.

Enfin, on cherche les meilleurs hyperparamètres **C** et **penalty**.

Hyperparamètres testés : - **C**: C'est l'inverse de la force de régularisation. Elle mesure la pénalité appliquée aux poids des variables. Plus **C** est petit, plus la régularisation est forte.

- **penalty(l1, l2)**: c'est le type de régularisation appliqué au modèle. Elle définit comment on punit la complexité pour éviter le surapprentissage.

Quand on choisit **l1 (Lasso)**, on demande au modèle d'essayer de trouver les variables les plus importantes et d'ignorer les autres en mettant leur poids à zéro.

Quand on choisit **l2 (Ridge)**, on demande au modèle de garder tous les coefficients mais de ne laisser aucun d'entre eux prendre plus d'importance. - **solver : saga**, c'est un paramètre qui permet au modèle de basculer librement entre les deux types de régularisation **l1** et **l2**.

- **class_weight : balanced**, c'est un paramètre qui permet de gérer le déséquilibre de la variable cible y . Concrètement, nous disons au modèle que faire une erreur sur un "yes" coûte beaucoup plus cher que de se tromper sur un "no".

- Régression logistique avec SMOTE

SMOTE est une méthode qui crée artificiellement de nouveaux exemples de la classe minoritaire.

SMOTE ne se contente pas de dupliquer les lignes existantes. Elle crée de nouveaux exemples synthétiques pour la classe minoritaire.

D'abord, elle choisit un point de la classe minoritaire. Ensuite, elle identifie ses k plus proches voisins. Enfin, elle crée un nouveau point n'importe où sur la ligne imaginaire qui relie le point initial à l'un de ses voisins.

Hyperparamètres testés : - `smote_k_neighbors` : les k plus proches voisins. - `modele_C` ou `C` - `modele_penalty` : ["l1", "l2"] - `solver` : saga

4.2 – Random Forest

Définition :

Mathématiquement, le Random Forest est une **agrégation** de plusieurs fonctions simples (arbres de décision).

- **Arbres de décision** $h(x)$: C'est une fonction qui sépare l'espace des données en "boîtes" rectangulaires par des conditions logiques.
- **La forêt** $H(x)$: Si on a T arbres, la prédiction finale $\hat{y}$, est le **vote majoritaire** de tous les arbres:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\} = H(x)$$

Comment il est appliqué dans notre cas?

D'abord, on encode les variables catégorielles. Les variables numériques restent telles quelles. Enfin, on teste plusieurs profondeurs et tailles de forêt.

Hyperparamètres testés : - `n_estimators` : le nombre d'arbres dans la forêt. Plus il y a d'arbres, plus le modèle est robuste. Cependant, au-delà d'un certain nombre, la performance stagne et le temps de calcul augmente.

- `max_depth` : la profondeur maximale de chaque arbre. Un arbre trop profond va apprendre les détails.
- `min_samples_split` : c'est le nombre minimal d'individus requis pour créer une nouvelle branche.
- `min_samples_leaf` : c'est le nombre minimal d'individus pour constituer une feuille finale.
- `max_features` : c'est le nombre de variables tirées au sort à chaque étape de construction de l'arbre.
- `class_weight` : balanced

4.3 – XGBoost (si disponible)

Définition :

XGBoost, eXtreme Gradient Boosting est une bibliothèque de machine learning qui utilise les arbres de décision auxquels est appliqué le *boosting* de gradient.

Le *boosting* de gradient est une méthode qui est utilisée pour réduire le nombre d'erreurs dans l'analyse prédictive des données.

XGBoost est donc un assemblage d'arbres décisionnels qui prédisent les résidus, et corrige les erreurs des arbres décisionnels précédents.

Le résidu représente l'erreur entre la valeur réelle et la valeur prédite.

Comment il est appliqué dans notre cas ?

On commence également par l'encodage des variables catégorielles.

Puis on calcule le ratio entre la classe négative et positive pour aider le modèle avec `scale_pos_weight`

Enfin, on teste plusieurs profondeurs, nombres d'arbres et taux d'apprentissage.

Hyperparamètres testés : - `n_estimators` - `max_depth` - `learning_rate` : c'est le "pas" d'apprentissage. C'est-à-dire à quel point on prend compte des corrections apportées par le nouvel arbre.

- `subsample` : c'est la fraction de lignes(individus), utilisée pour chaque arbre.

- `colsample_bytree` : c'est la fraction de colonnes(variables), utilisée par chaque arbre.

`scale_pos_weight` : permet de mesurer le poids accordé à la classe positive.

4.4 – Construisons les préprocesseurs

```
preprocesseur_avec_scaler <- list(
  variables_numeriques = colonnes_numeriques,
  variables_categorielles = colonnes_categorielles,
  standardisation = TRUE,
  encodage = "one_hot"
)

preprocesseur_sans_scaler <- list(
  variables_numeriques = colonnes_numeriques,
  variables_categorielles = colonnes_categorielles,
  standardisation = FALSE,
  encodage = "one_hot"
)

cat("Préprocesseurs créés.
")
```

Préprocesseurs créés.

4.5 – Calculons le ratio de déséquilibre sur le train

```
nb_non <- sum(y_train == 0)
nb_oui <- sum(y_train == 1)
ratio_positif <- nb_non / nb_oui

cat("Nombre de no sur le train :", nb_non, "
")
```

Nombre de no sur le train : 27216

```
cat("Nombre de yes sur le train :", nb_oui, "
")
```

Nombre de yes sur le train : 1607

```
cat("Ratio no/yes :", round(ratio_positif, 2), "
")
```

Ratio no/yes : 16.94

4.6 – Construisons les pipelines des modèles

```
pipeline_logistique <- list(
  preparation = preprocesseur_avec_scaler,
  modele = "regression_logistique_ponderee"
)

pipeline_logistique_smote <- list(
  preparation = preprocesseur_avec_scaler,
  reequilibrage = "SMOTE",
  modele = "regression_logistique"
)

pipeline_rf <- list(
  preparation = preprocesseur_sans_scaler,
  modele = "random_forest"
```

```

)
modeles <- list(
  logistique_class_weight = list(
    pipeline = pipeline_logistique,
    parametres = tidyr::crossing(
      `modele__C` = c(0.001, 0.01, 0.1, 1, 10),
      `modele__penalty` = c("l1", "l2")
    ),
    type_recherche = "grid"
  ),
  logistique_smote = list(
    pipeline = pipeline_logistique_smote,
    parametres = tidyr::crossing(
      `smote__k_neighbors` = c(3, 5, 7),
      `modele__C` = c(0.01, 0.1, 1),
      `modele__penalty` = c("l1", "l2")
    ),
    type_recherche = "grid"
  ),
  random_forest = list(
    pipeline = pipeline_rf,
    parametres = tidyr::crossing(
      `modele__n_estimators` = c(200, 400),
      `modele__max_depth` = c(5, 10, 20, NA),
      `modele__min_samples_split` = c(2, 5, 10),
      `modele__min_samples_leaf` = c(1, 2, 5),
      `modele__max_features` = c("sqrt", "0.5")
    ),
    type_recherche = "random",
    n_iter = 12
  )
)

if (xgboost_disponible) {
  pipeline_xgb <- list(
    preparation = preprocesseur_sans_scaler,
    modele = "xgboost"
  )

  modeles$xgboost <- list(
    pipeline = pipeline_xgb,
    parametres = tidyr::crossing(
      `modele__n_estimators` = c(100, 200),
      `modele__max_depth` = c(3, 5, 7),
      `modele__learning_rate` = c(0.03, 0.1),
      `modele__subsample` = c(0.8, 1.0),
      `modele__colsample_bytree` = c(0.8, 1.0)
    ),
    type_recherche = "random",
    n_iter = 10
  )
}

cat("Modèles construits :", paste(names(modeles), collapse = ", "), "\n")

```

Modèles construits : logistique_class_weight, logistique_smote, random_forest, xgboost

4.7 – Récapitulatif des espaces d’hyperparamètres

```
lignes_hyper <- purrr::imap_dfr(modeles, function(infos, nom_modele) {
  tibble(
    modele = nom_modele,
    hyperparametre = names(infos$parametres),
    valeurs_testees = purrr::map_chr(infos$parametres, resume_valeurs_uniques)
  )
})

tableau_hyper <- lignes_hyper
afficher_tableau_pdf(
  tableau_hyper,
  colonnes_a_enrouler = c(2, 3),
  largeurs = c("3cm", "4.2cm", "7cm"),
  font_size = 7,
  col_names = c("modèle", "hyperparamètre", "valeurs testées")
)
```

modèle	hyperparamètre	valeurs testées
logistique_class_weight	modele__ C	0.001, 0.01, 0.1, 1, 10
logistique_class_weight	modele__ penalty	11, 12
logistique_smote	smote__ k_neighbors	3, 5, 7
logistique_smote	modele__ C	0.01, 0.1, 1
logistique_smote	modele__ penalty	11, 12
random_forest	modele__ n_estimators	200, 400
random_forest	modele__ max_depth	5, 10, 20, NA
random_forest	modele__ min_samples_split	2, 5, 10
random_forest	modele__ min_samples_leaf	1, 2, 5
random_forest	modele__ max_features	0.5, sqrt
xgboost	modele__ n_estimators	100, 200
xgboost	modele__ max_depth	3, 5, 7
xgboost	modele__ learning_rate	0.03, 0.1
xgboost	modele__ subsample	0.8, 1
xgboost	modele__ colsample_bytree	0.8, 1

4.8 – Optimisons les hyperparamètres sur le jeu d'entraînement

```
splits_cv <- creer_splits_temporels(nrow(train), n_splits = 3)

resultats_tuning <- list()
objets_recherche <- list()
lignes_tuning <- list()

for (nom_modele in names(modeles)) {
  cat(strrep("=", 70), "
")
  cat("Optimisation du modèle :", nom_modele, "
")

  debut <- Sys.time()

  optimisation <- optimiser_modele(
    nom_modele = nom_modele,
    infos = modeles[[nom_modele]],
    train_df = train,
    splits = splits_cv,
    colonnes_numeriques = colonnes_numeriques,
    colonnes_categorielles = colonnes_categorielles,
    ratio_positif = ratio_positif
  )

  duree <- round(as.numeric(difftime(Sys.time(), debut, units = "secs")), 2)
```



```

resultats_tuning[[nom_modele]] <- optimisation$details
objets_recherche[[nom_modele]] <- optimisation

lignes_tuning[[nom_modele]] <- tibble(
  modele = nom_modele,
  methode_recherche = modeles[[nom_modele]]$type_recherche,
  pr_auc_cv_train = optimisation$best_score,
  meilleurs_parametres = paste(
    names(optimisation$best_params),
    unlist(optimisation$best_params),
    sep = "=",
    collapse = " ; "
  ),
  duree_secondes = duree
)

cat("Meilleur score CV (PR-AUC) :", round(optimisation$best_score, 4), "
")
cat("Meilleurs paramètres :
")
print(optimisation$best_params)
cat("Durée (s) :", duree, "
")
}

## =====
## Optimisation du modèle : logistique_class_weight
## Meilleur score CV (PR-AUC) : 0.0747
## Meilleurs paramètres :
## $modele__C
## [1] 0.01
##
## $modele__penalty
## [1] "l2"
##
## Durée (s) : 15.61
## =====
## Optimisation du modèle : logistique_smote
## Meilleur score CV (PR-AUC) : 0.0597
## Meilleurs paramètres :
## $smote__k_neighbors
## [1] 3
##
## $modele__C
## [1] 0.1
##
## $modele__penalty
## [1] "l1"
##
## Durée (s) : 38.91
## =====
## Optimisation du modèle : random_forest
## Meilleur score CV (PR-AUC) : 0.0725
## Meilleurs paramètres :
## $modele__n_estimators

```

```
## [1] 200
##
## $modele__max_depth
## [1] 5
##
## $modele__min_samples_split
## [1] 10
##
## $modele__min_samples_leaf
## [1] 5
##
## $modele__max_features
## [1] "0.5"
##
## Durée (s) : 92.48
## =====
## Optimisation du modèle : xgboost
## Meilleur score CV (PR-AUC) : 0.0606
## Meilleurs paramètres :
## $modele__n_estimators
## [1] 100
##
## $modele__max_depth
## [1] 7
##
## $modele__learning_rate
## [1] 0.03
##
## $modele__subsample
## [1] 1
##
## $modele__colsample_bytree
## [1] 1
##
## Durée (s) : 30.46

tableau_tuning <- bind_rows(lignes_tuning) %>%
  arrange(desc(pr_auc_cv_train)) %>%
  mutate(pr_auc_cv_train = round(pr_auc_cv_train, 6))

afficher_tableau_pdf(
  tableau_tuning,
  colonnes_a_enrouler = c(1, 4),
  largeurs = c("2.6cm", "1.8cm", "1.6cm", "6.0cm", "1.4cm"),
  font_size = 6,
  col_names = c("modèle", "recherche", "PR-AUC CV", "meilleurs paramètres", "durée (s)")
)
```

- Ici, on choisit les meilleurs hyperparamètres uniquement sur le **train**.
- La métrique de comparaison pendant cette phase est la **PR-AUC** au sens de l'**Average Precision** de **scikit-learn**, car notre jeu de données est déséquilibré.

4.9 – Mettons maintenant les modèles en compétition sur le jeu de validation

```
resultats_validation <- list()

for (nom_modele in names(objets_recherche)) {
```

modèle	recherche	PR-AUC CV	meilleurs paramètres	durée (s)
logistique__class__weight	grid	0.074698	modele__ C = 0.01 ; modele__ penalty = l2	15.61
random__forest	random	0.072532	modele__ n__ estimators = 200 ; modele__ max__ depth = 5 ; modele__ min__ samples__ split = 10 ; modele__ min__ samples__ leaf = 5 ; modele__ max__ features = 0.5	92.48
xgboost	random	0.060647	modele__ n__ estimators = 100 ; modele__ max__ depth = 7 ; modele__ learning__ rate = 0.03 ; modele__ subsample = 1 ; modele__ colsample__ bytree = 1	30.46
logistique__smote	grid	0.059708	smote__ k__ neighbors = 3 ; modele__ C = 0.1 ; modele__ penalty = l1	38.91

```

meilleur_modele <- objets_recherche[[nom_modele]]$best_model

y_proba_valid <- predire_proba(meilleur_modele, valid)
y_pred_valid_05 <- ifelse(y_proba_valid >= 0.50, 1, 0)

pr_auc_valid <- calculer_pr_auc(y_valid, y_proba_valid)
roc_auc_valid <- calculer_roc_auc(y_valid, y_proba_valid)
precision_valid <- calculer_precision(y_valid, y_pred_valid_05)
recall_valid <- calculer_recall(y_valid, y_pred_valid_05)
f1_valid <- calculer_f1(y_valid, y_pred_valid_05)

metriques_top20 <- calculer_lift_recall_top20(y_valid, y_proba_valid, proportion = 0.20)

resultats_validation[[nom_modele]] <- tibble(
  modele = nom_modele,
  pr_auc_validation = pr_auc_valid,
  roc_auc_validation = roc_auc_valid,
  precision_validation_0_5 = precision_valid,
  recall_validation_0_5 = recall_valid,
  f1_validation_0_5 = f1_valid,
  lift_top_20_validation = metriques_top20$lift_top_20,
  recall_top_20_validation = metriques_top20$recall_top_20,
  meilleurs_parametres = paste(
    names(objets_recherche[[nom_modele]]$best_params),
    unlist(objets_recherche[[nom_modele]]$best_params),
    sep = "=",
    collapse = " ; "
  )
)
}

tableau_validation <- bind_rows(resultats_validation) %>%
  arrange(desc(pr_auc_validation)) %>%
  mutate(across(where(is.numeric), ~ round(.x, 4)))

tableau_validation_affichage <- tableau_validation %>%
  transmute(
    modele,
    pr_auc = pr_auc_validation,
    roc_auc = roc_auc_validation,
    precision = precision_validation_0_5,
    recall = recall_validation_0_5,
    f1 = f1_validation_0_5,
    lift_20 = lift_top_20_validation,
    recall_20 = recall_top_20_validation,
    meilleurs_parametres
  )

```

```

afficher_tableau_pdf(
  tableau_validation_affichage,
  colonnes_a_enrouler = c(1, 9),
  largeurs = c("1.8cm", "0.9cm", "0.9cm", "0.9cm", "0.9cm", "0.8cm", "1.1cm", "1.2cm", "4.0cm"),
  font_size = 5,
  col_names = c("modèle", "PR-AUC", "ROC-AUC", "préc.", "recall", "F1", "Lift@20%", "Recall@20%",
    ↵ "meilleurs paramètres")
)

```

modèle	PR-AUC	ROC-AUC	préc.	recall	F1	Lift@20%	Recall@20%	meilleurs paramètres
logistique__smote	0.1585	0.6158	0.1094	0.9817	0.1968	1.5971	0.3196	smote__ k_neighbors = 3 ; modele__ C = 0.1 ; modele__ penalty = l1
logistique__class_weight	0.1556	0.6114	0.1066	1.0000	0.1927	1.4526	0.2907	modele__ C = 0.01 ; modele__ penalty = l2
random__forest	0.1322	0.5287	0.0000	0.0000	0.0000	1.2549	0.2511	modele__ n_estimators = 200 ; modele__ max_depth = 5 ; modele__ min_samples_split = 10 ; modele__ min_samples_leaf = 5 ; modele__ max_features = 0.5
xgboost	0.1217	0.5337	0.1113	0.7900	0.1952	1.2549	0.2511	modele__ n_estimators = 100 ; modele__ max_depth = 7 ; modele__ learning_rate = 0.03 ; modele__ subsample = 1 ; modele__ __ colsample_bytree = 1

Le **meilleur modèle** selon la métrique **PR-AUC** est celui classé en première position dans le tableau ci-dessus.

4.10 – Identifions le modèle champion

```

modele_champion_nom <- tableau_validation$modele[1]
modele_champion_train <- objets_recherche[[modele_champion_nom]]$best_model

cat("Le modèle champion à ce stade est :", modele_champion_nom, "
")

```

```
## Le modèle champion à ce stade est : logistique_smote
```

```
cat("Ses meilleurs hyperparamètres sont :
")
```

```
## Ses meilleurs hyperparamètres sont :
```

```
print(objets_recherche[[modele_champion_nom]]$best_params)
```

```

## $smote__k_neighbors
## [1] 3
##
## $modele__C
## [1] 0.1
##
## $modele__penalty
## [1] "l1"

```

Le **modèle champion** est : **logistique__smote**. Il a été retenu car il obtient la meilleure **PR-AUC** sur le jeu de validation. Dans ce projet, cette métrique est la plus adaptée car la classe positive est minoritaire et nous voulons comparer les modèles sur leur capacité globale à bien classer les clients susceptibles de souscrire, sans dépendre d'un seul seuil fixe.

ETAPE 5: EVALUER LE MODELE

Nous avons choisi le meilleur algorithme. Il faut maintenant : 1. choisir un **seuil de décision** sur la validation ; 2. réentraîner le champion sur **train + validation** ; 3. évaluer une seule fois sur le **test** final.

5.1 – Choisissons le seuil de décision sur la validation

```
y_proba_valid_champion <- predire_proba(modele_champion_train, valid)

seuils_optimises <- choisir_seuil_f1(y_valid, y_proba_valid_champion)
seuil_retenu <- seuils_optimises$seuil
tableau_seuils <- seuils_optimises$tableau

cat("Seuil retenu :", round(seuil_retenu, 4), "
")

## Seuil retenu : 0.7615

afficher_tableau_pdf(
  head(tableau_seuils, 10) %>% mutate(across(everything(), ~ round(.x, 4))),
  font_size = 8,
  col_names = c("seuil", "précision", "recall", "F1")
)
```

seuil	précision	recall	F1
0.7615	0.1573	0.4323	0.2306
0.7615	0.1572	0.4323	0.2305
0.7625	0.1579	0.4262	0.2305
0.7615	0.1570	0.4323	0.2303
0.7673	0.1627	0.3942	0.2303
0.7625	0.1577	0.4262	0.2303
0.7614	0.1569	0.4323	0.2302
0.7673	0.1626	0.3942	0.2302
0.7625	0.1579	0.4247	0.2302
0.7616	0.1570	0.4307	0.2302

Le seuil retenu est celui qui **maximise le F1-score** sur le jeu de validation.

5.2 – Réentraînons le modèle champion sur train + validation

```
X_train_valid <- bind_rows(X_train, X_valid)
y_train_valid <- c(y_train, y_valid)

train_valid <- bind_rows(train, valid)

modele_final <- ajuster_modele(
  nom_modele = modele_champion_nom,
  train_df = train_valid,
  params = objets_recherche[[modele_champion_nom]]$best_params,
  colonnes_numeriques = colonnes_numeriques,
  colonnes_categorielles = colonnes_categorielles,
  ratio_positif = ratio_positif
)

cat("Le modèle final a été réentraîné sur train + validation.
")
```

Le modèle final a été réentraîné sur train + validation.

5.3 – Évaluons le modèle final sur le test

```

y_proba_test <- predire_proba(modele_final, test)
y_pred_test <- ifelse(y_proba_test >= seuil_retenu, 1, 0)

pr_auc_test <- calculer_pr_auc(y_test, y_proba_test)
roc_auc_test <- calculer_roc_auc(y_test, y_proba_test)
precision_test <- calculer_precision(y_test, y_pred_test)
recall_test <- calculer_recall(y_test, y_pred_test)
f1_test <- calculer_f1(y_test, y_pred_test)

matrice_conf <- table(
  Reference = factor(y_test, levels = c(0, 1)),
  Prediction = factor(y_pred_test, levels = c(0, 1))
)

metriques_top20_test <- calculer_lift_recall_top20(y_test, y_proba_test, proportion = 0.20)
lift_top_20_test <- metriques_top20_test$lift_top_20
recall_top_20_test <- metriques_top20_test$recall_top_20

tableau_test <- tibble(
  modele_retenu = modele_champion_nom,
  seuil_retenu = seuil_retenu,
  pr_auc_test = pr_auc_test,
  roc_auc_test = roc_auc_test,
  precision_test = precision_test,
  recall_test = recall_test,
  f1_test = f1_test,
  lift_top_20_test = lift_top_20_test,
  recall_top_20_test = recall_top_20_test
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 4)))

afficher_tableau_pdf(
  tableau_test,
  colonnes_a_enrouler = 1,
  largeurs = c("2.6cm", "1.1cm", "1.1cm", "1.1cm", "1.2cm", "1.1cm", "0.9cm", "1.3cm", "1.4cm"),
  font_size = 6,
  col_names = c("modèle retenu", "seuil", "PR-AUC", "ROC-AUC", "précision", "recall", "F1",
    ↪ "Lift@20%", "Recall@20%")
)

```

modèle retenu	seuil	PR-AUC	ROC-AUC	précision	recall	F1	Lift@20%	Recall@20%
logistique_ smote	0.7615	0.5507	0.6621	0.4433	0.9368	0.6018	1.4561	0.2914

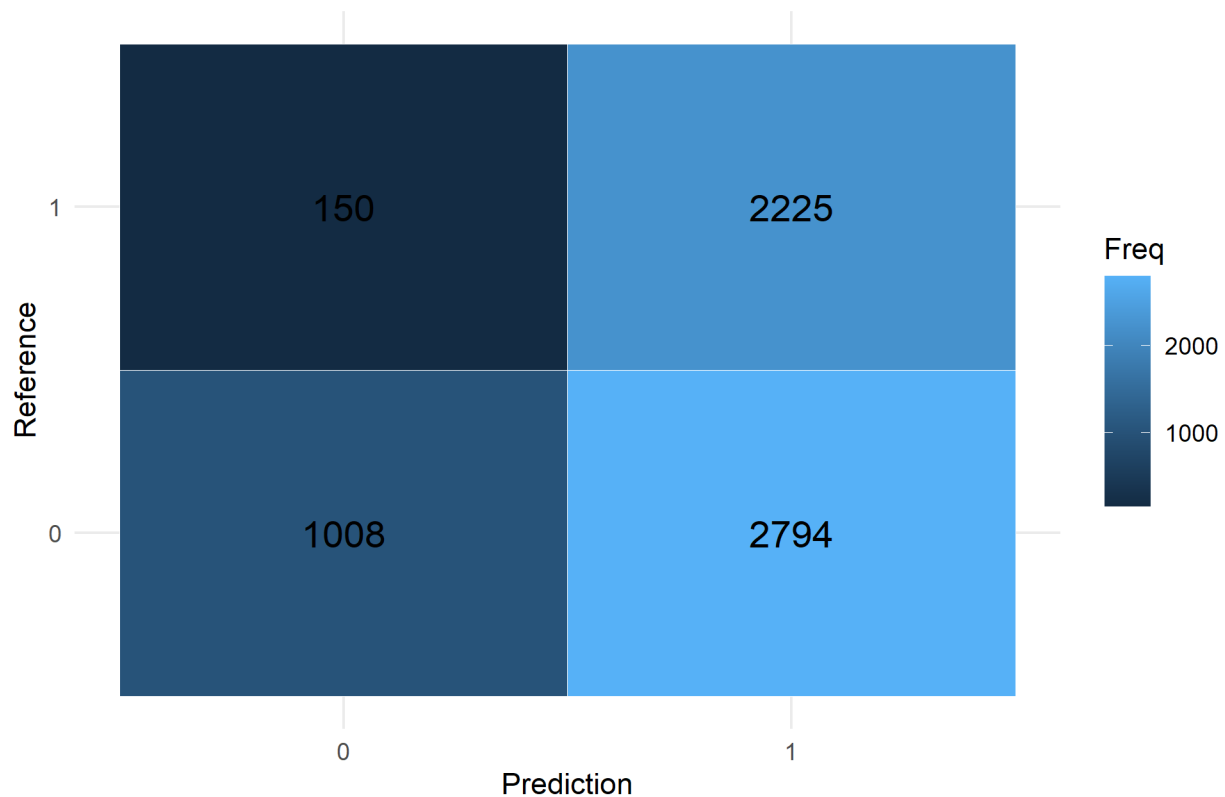
5.4 – Affichons la matrice de confusion

```

as.data.frame(matrice_conf) %>%
  ggplot(aes(x = Prediction, y = Reference, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 5) +
  labs(title = "Matrice de confusion - modèle final") +
  theme_minimal()

```

Matrice de confusion - modèle final



5.5 – Conclusion

```
cat("MODÈLE FINAL RETENU :", modele_champion_nom, "
")

## MODÈLE FINAL RETENU : logistique_smote

cat("MEILLEURS HYPERPARAMÈTRES :
")

## MEILLEURS HYPERPARAMÈTRES :

print(objets_recherche[[modele_champion_nom]]$best_params)

## $smote__k_neighbors
## [1] 3
##
## $modele__C
## [1] 0.1
##
## $modele__penalty
## [1] "l1"

cat("SEUIL FINAL RETENU :", round(seuil_retenu, 4), "
")

## SEUIL FINAL RETENU : 0.7615
```

```

cat("PR-AUC test :", round(pr_auc_test, 4), "
")

## PR-AUC test : 0.5507

cat("ROC-AUC test :", round(roc_auc_test, 4), "
")

## ROC-AUC test : 0.6621

cat("Precision test :", round(precision_test, 4), "
")

## Precision test : 0.4433

cat("Recall test :", round(recall_test, 4), "
")

## Recall test : 0.9368

cat("F1 test :", round(f1_test, 4), "
")

## F1 test : 0.6018

cat("Lift@20% test :", round(lift_top_20_test, 4), "
")

## Lift@20% test : 1.4561

cat("Recall@20% test :", round(recall_top_20_test, 4), "
")

## Recall@20% test : 0.2914

```

Conclusion finale

Le modèle final retenu est **logistique_smote**. Nous avons opté pour une approche équilibrée dans ce projet, c'est-à-dire que l'objectif n'est pas seulement de maximiser le **nombre de clients détectés**, ni de **réduire les faux positifs** uniquement, mais de trouver un compromis raisonnable entre les deux.

Les résultats obtenus sur le jeu de test montrent ce compromis. En **précision**, nous obtenons **0.4433**. Cela signifie que, parmi les clients prédits comme souscripteurs, cette proportion souscrit réellement.

En **recall**, nous obtenons **0.9368**. Cela signifie que le modèle retrouve cette proportion des clients qui auraient réellement souscrit.

Enfin, le **F1-score** vaut **0.6018**. Cette valeur résume l'équilibre entre **précision** et **rappel** au seuil retenu. Pour la lecture métier, nous complétons cette analyse avec **Lift@20% = 1.4561** et **Recall@20% = 0.2914**.